

A Post-Quantum Accountable Sanitizable Signature Scheme Based on Unbalanced Oil and Vinegar

Zhiwei Wang

Nanjing University of Posts and Telecommunications
zhwwang@njupt.edu.cn

Abstract. Sanitizable signature schemes (SSS) allow a designated sanitizer to modify admissible portions of a signed message while preserving the validity of the original signer’s authorisation. All existing SSS constructions satisfying the Brzuska et al. security framework rely on classical number-theoretic assumptions broken by Shor’s algorithm. We present **UOV-San**, a sanitizable signature scheme based entirely on multivariate cryptography. The construction employs a dual-signature architecture with strict key separation enforced by a two-message interactive signing protocol: the signer holds only sk_S and the public trapdoor-hash key ck , whereas the sanitizer holds sk_{San} and the trapdoor key tk . We introduce a target second-preimage resistance assumption, **tSPR**, for UOV public maps. It captures the hardness of finding a distinct input $r' \neq r$ that collides with a given random target input r under the public map P , namely $P(r') = P(r)$. Based on this assumption we construct **MV-CH**, a dedicated multivariate target-collision trapdoor hash. **MV-CH** is not a standard message-dependent chameleon hash of the form $CH(m, r)$; instead, it hashes only the randomness r , while message binding is provided by the upper-layer dual-signature construction. The sanitizer can use the UOV trapdoor to generate target collisions, whereas outsiders face the **tSPR** problem, except for random-oracle target-collision events. **UOV-San** provably achieves unforgeability, immutability, and accountability—including against a malicious signer—under UOV EUF-CMA security and **tSPR** in the random-oracle model. We forgo transparency and privacy: these properties are structurally incompatible with the dual-signature architecture and key-separation requirement, and are operationally unnecessary for our target application domains such as supply-chain audit, government document redaction, and blockchain audit trails. Experimental evaluation confirms practical signing times under 5 ms and verification times under 2 ms on commodity hardware.

Keywords. Post-quantum cryptography; Sanitizable signatures; UOV; Multivariate target-collision trapdoor hash; Target second-preimage resistance; **tSPR**; Key separation; Interactive signing; Accountability

1 Introduction

Sanitizable signature schemes (SSS), introduced by Ateniese et al. [2], allow a pre-designated sanitizer to modify admissible message blocks while keeping the original signer’s authorisation valid. The security framework of Brzuska et al. [6] formalises five properties: unforgeability, immutability, privacy, transparency, and accountability. SSS have found applications in electronic health records [7], vehicular communications [14], and redactable blockchains [24].

1.1 The Post-Quantum Gap

Shor’s algorithm [22] breaks RSA, discrete-logarithm, and elliptic-curve assumptions in polynomial quantum time, rendering all classical SSS constructions insecure against a quantum adversary. NIST’s post-quantum standardisation process [20] has produced lattice-based and hash-based standards (ML-DSA, SLH-DSA), but no post-quantum SSS has been standardised. The only prior post-quantum SSS is the code-based construction of Ahmad et al. [1] (~ 4 KB signatures at 128-bit security), which relies on the syndrome decoding problem. Multivariate cryptography, particularly UOV [15], offers an attractive alternative: 25 years of cryptanalytic confidence [4,16], compact signatures (~ 116 bytes at Level I), and fast verification.

1.2 Related Work

Sanitizable signature schemes. Ateniese et al. [2] introduced SSS with five informal security properties. Brzuska et al. [6] provided the first rigorous formalisation and proved that transparency and accountability are mutually exclusive without a trusted third party in the standard model. Subsequent work extended the framework: Brzuska et al. [7] introduced unlinkability, a strong form of privacy preventing linking of sanitized versions; Camenisch et al. [8] proposed invisible sanitizable signatures hiding which blocks are admissible; Lai et al. [19] constructed SSS from lattices but with large parameters and without accountability; and Krenn et al. [18] studied SSS in the universal composability framework. Redactable signatures, a related primitive allowing deletion but not substitution of blocks, have been studied extensively [26,11].

Multivariate cryptography. The Oil-and-Vinegar (OV) scheme was proposed by Patarin [21] and broken by Kipnis and Shamir [16]. The Unbalanced Oil-and-Vinegar (UOV) variant [15] with $v \geq o$ has resisted all known attacks for over 25 years. Rainbow [12], a layered variant, was broken by Beullens [4] in 2022. UOV remains unbroken and was submitted to NIST’s additional signatures process [27]. Other multivariate signature schemes include GeMSS [9], though these have been subject to ongoing cryptanalysis.

Chameleon hashes and target-collision trapdoor hashes. Chameleon hashes were introduced by Krawczyk and Rabin [17] and have been used in sanitizable signatures since [2]. Standard chameleon hashes usually provide an interface $CH(m, r)$, where the trapdoor holder can adapt r when the message m changes. Our construction uses a different and more specialised primitive. The MV-CH component introduced in this paper is a target-collision trapdoor hash: it hashes only a randomness value r , and the trapdoor holder can find $r' \neq r$ such that the hash value is preserved. Message binding is provided outside MV-CH by the dual-signature layer. This distinction is important because MV-CH is not intended to satisfy arbitrary collision resistance or the full syntax of a standard message-dependent chameleon hash. Its security notion is target second-preimage resistance: given a random r , it should be infeasible to find $r' \neq r$ with $P(r') = P(r)$ under the UOV public map P .

1.3 Three Compounding Challenges of SSS

Challenge 1: Correctness–immutability tension. A single-signature approach cannot satisfy both correctness and immutability. Our resolution: a *dual-signature architecture* where σ_S covers only sanitization-invariant chameleon commitments and σ_{San} covers full message content.

Challenge 2: Implementable multivariate target-collision trapdoor hash. Our resolution is MV-CH, defined by

$$\text{TCTH.Hash}(ck, r) := H_2(P(r)).$$

It depends only on the randomness r . Trapdoor collision generation reduces to standard UOV signing with target $P(r)$, and no random-oracle programming is required. MV-CH is deliberately not a standard message-dependent chameleon hash; message binding is provided by the Layer 2 signature.

Challenge 3: Key separation and true accountability. Our resolution: a *two-message interactive signing protocol* where the signer produces σ_S using only sk_S , and the sanitizer produces σ_{San} using only sk_{San} . Neither party ever holds the other’s private key.

1.4 Security Properties: Achieved and Foregone

Achieved properties. UOV-San achieves unforgeability, immutability, and accountability with complete proofs in the random oracle model. Accountability holds even against a malicious signer who attempts to frame the sanitizer.

Foregone properties. UOV-San does not achieve *transparency* or *privacy*. We forgo these properties because: (i) they are structurally incompatible with the dual-signature architecture required for key separation (see Section 7); and (ii) they are operationally unnecessary for our target application domains. We do not claim these properties and do not attempt to prove them.

1.5 Our Contributions

Our contributions include a key-separated dual-signature architecture, a two-message interactive signing protocol, true accountability against a malicious signer, complete random-oracle proofs, an implementable multivariate target-collision trapdoor hash MV-CH with

$$\text{TCTH.Hash}(ck, r) := H_2(P(r)),$$

the tSPR assumption for UOV public maps, and experimental validation.

1.6 Paper Organisation

Section 2 reviews preliminaries. Section 3 defines SSS syntax and security. Section 4 presents UOV in detail. Section 5 introduces tSPR and constructs MV-CH. Section 6 presents UOV-San. Section 7 discusses application scope and the deliberate choice to forgo transparency and privacy. Section 8 contains security proofs. Section 9 presents experimental results. Section 10 covers parameters and efficiency. Section 11 concludes.

2 Preliminaries

2.1 Notation

λ : security parameter. $x \xleftarrow{\$} S$: uniform sampling from set S . $\text{negl}(\lambda)$: a function negligible in λ . $[\ell] = \{1, \dots, \ell\}$. For a message $m = (m_1, \dots, m_\ell)$ and admissibility set $\text{ADM} \subseteq [\ell]$, write $m_{\overline{\text{ADM}}} = m_{\overline{\text{ADM}}}$ for the sub-vector of *fixed* (non-admissible) blocks, and $(m_i)_{i \in \text{ADM}}$ for the *admissible* blocks.

2.2 Finite Fields and Multivariate Quadratic Maps

Let $q = 2^k$ for a fixed positive integer k (e.g., $k = 8$, $q = 256$). We emphasise that q is a *fixed* scheme parameter, not a function of λ ; security is achieved by choosing the number of oil variables o and vinegar variables v large enough relative to λ .

An *MQ map* is $P = (p_1, \dots, p_o) : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^o$ where each $p_j : \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ is a quadratic polynomial over $\mathbb{F}_q$. The *MQ problem* asks: given P and $\mathbf{y} \in \mathbb{F}_q^o$, find $\mathbf{x} \in \mathbb{F}_q^n$ with $P(\mathbf{x}) = \mathbf{y}$. The MQ problem is NP-hard [13] and conjectured hard on average [3, 10].

2.3 Strong EUF-CMA Security

Definition 1 (sEUF-CMA). A signature scheme $\Sigma = (\text{KG}, \text{Sg}, \text{Vf})$ is strongly EUF-CMA secure if for every PPT adversary $\mathcal{A}$:

$$\text{Adv}_{\Sigma, \mathcal{A}}^{\text{sEUF-CMA}}(\lambda) := \Pr \left[\begin{array}{l} (pk, sk) \leftarrow \text{KG}(1^\lambda); \\ (m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sg}}}(pk) : \\ \text{Vf}(pk, m^*, \sigma^*) = 1 \wedge (m^*, \sigma^*) \notin Q_{\text{Sg}} \end{array} \right] = \text{negl}(\lambda),$$

where Q_{Sg} records all (message, signature) pairs returned by $\mathcal{O}_{\text{Sg}}$.

Assumption 1 (UOV signing security). *For the UOV parameters used in this work (Table 1), we assume:*

1. **EUF-CMA security** of hash-and-sign UOV for fresh message targets;
2. **Target second-preimage resistance** of the UOV public map: given a valid signature σ on a target $h = P(\sigma)$, it is infeasible to find $\sigma' \neq \sigma$ such that $P(\sigma') = h$.

Together these two assumptions imply the strong unforgeability property (sEUF-CMA) needed in our proofs. See Remarks 2 and [27,4].

2.4 Target-Collision Trapdoor Hashes

We use a specialised trapdoor primitive rather than a standard message-dependent chameleon hash. In a standard chameleon hash one typically evaluates $CH(m, r)$, and the trapdoor holder can adapt the randomness when the message m changes. Our construction requires only a weaker and more focused functionality: for a publicly verifiable hash of a randomness value r , the trapdoor holder can find another randomness value $r' \neq r$ producing the same hash. The message itself is bound by the higher-level signature layer.

Definition 2 (Target-Collision Trapdoor Hash). *A target-collision trapdoor hash, abbreviated TCTH, is a tuple of algorithms*

$$\text{TCTH} = (\text{TCTH.KeyGen}, \text{TCTH.}, \text{TCTH.})$$

with the following syntax.

- $\text{TCTH.KeyGen}(1^\lambda) \rightarrow (ck, tk)$: generates a public hash key ck and a trapdoor key tk .
- $\text{TCTH.}(ck, r) \rightarrow c$: on input $r \in \mathbb{F}_q^n$, outputs a public digest c .
- $\text{TCTH.}(tk, r) \rightarrow r'$: using the trapdoor, outputs $r' \in \mathbb{F}_q^n$ such that $r' \neq r$ and

$$\text{TCTH.}(ck, r') = \text{TCTH.}(ck, r).$$

The primitive satisfies the following properties.

- **Correctness.** *For honestly generated keys and all $r \in \mathbb{F}_q^n$, except with negligible probability over the randomness of TCTH. , the output r' satisfies $r' \neq r$ and $\text{TCTH.}(ck, r') = \text{TCTH.}(ck, r)$.*
- **Target second-preimage resistance.** *For every efficient adversary $\mathcal{A}$ without tk ,*

$$\text{TSPR}_{\text{TCTH}, \mathcal{A}}(\lambda) := \Pr \left[\begin{array}{l} (ck, tk) \leftarrow \text{TCTH.KeyGen}(1^\lambda); \\ r \xleftarrow{\$} \mathbb{F}_q^n; \\ r' \leftarrow \mathcal{A}(ck, r) : \\ r' \neq r \wedge \text{TCTH.}(ck, r') = \text{TCTH.}(ck, r) \end{array} \right]$$

is negligible.

- Trapdoor collision generation. *Given tk , the algorithm TCTH. runs in expected polynomial time.*

We do not require arbitrary collision resistance. In fact, arbitrary collisions of quadratic maps can be found efficiently by differential methods when the adversary is allowed to choose both inputs. This is not the security notion needed in UOV-San: after signing, the adversary is given a fixed target randomness r embedded in a valid signature and must find a second preimage for that target.

2.5 Hash to Finite Field Vector

Define $\text{HashToF} : \{0, 1\}^* \rightarrow \mathbb{F}_q^o$ as a random oracle, instantiated as $\text{HashToF}(x) := F_q(\text{SHAKE256}(x, o \cdot \log_2 q))$. We use four independent random oracles H_S, H_{San}, H_2, H_J , each outputting $\{0, 1\}^\kappa$ (with HashToF applied when $\mathbb{F}_q^o$ needed).

3 Sanitizable Signature Schemes

3.1 Syntax

We follow the syntax of Brzuska et al. [6] with the modification that **Sign** is an interactive protocol enforcing key separation.

KeyGen(1^λ): Generates key material for both parties. Outputs (pk_S, sk_S) for the signer and $(pk_{San}, sk_{San}, ck, tk)$ for the sanitizer, with ck public and tk held only by the sanitizer.

Sign: An interactive two-message protocol between signer and sanitizer:

- $\text{Sign}_1(sk_S, pk_{San}, ck, m, \text{ADM}) \rightarrow (\text{msg}_1, \text{st})$: signer's first message, using only sk_S and public ck .
- $\text{Sign}_2(sk_{San}, pk_S, pk_{San}, ck, \text{msg}_1) \rightarrow \text{msg}_2$: sanitizer's response, using only sk_{San} ; does not use tk .
- $\text{Sign}_3(sk_S, \text{st}, \text{msg}_2) \rightarrow \sigma$: signer assembles the final signature and creates the accountability tag.

Verify($pk_S, pk_{San}, ck, m, \text{ADM}, \sigma$): Outputs 1 (accept) or 0 (reject). Uses only public keys.

Sanitize($sk_{San}, tk, ck, pk_S, pk_{San}, m, \text{ADM}, \sigma, \text{MOD}$): The sanitizer modifies admissible blocks according to MOD using sk_{San} and tk . Outputs (m', σ') .

Proof(m, ADM, σ): Extracts the embedded accountability tag from σ . Outputs proof $\pi := (\omega, \tau)$.

Judge($pk_S, pk_{San}, m, \text{ADM}, \sigma$): Determines the origin of the current signature version by verifying the embedded tag. Outputs **Signer** or **Sanitizer** or $\perp$.

Key separation invariant: signer uses only sk_S and ck ; sanitizer uses sk_{San} and tk .

3.2 Security Oracles

$\mathcal{O}_{\text{Sign}}(m, \text{ADM})$: Runs the full interactive protocol (simulator holds both keys), records (m, ADM, σ) in Q , returns σ .

$\mathcal{O}_{\text{San}}(m, \text{ADM}, \sigma, \text{MOD})$: Verifies $(m, \text{ADM}, \sigma) \in Q$, runs **Sanitize**, records $(m', \text{ADM}, \sigma')$ in Q , returns (m', σ') .

3.3 Security Definitions

Definition 3 (Unforgeability [6]). UOV-San is unforgeable if for every PPT adversary $\mathcal{A}$ without sk_S and sk_{San} :

$$\text{Adv}_{\text{UOV-San}, \mathcal{A}}^{\text{UNF}}(\lambda) := \Pr \left[\begin{array}{l} (m^*, \text{ADM}^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}, \mathcal{O}_{\text{San}}}(pk_S, pk_{\text{San}}) : \\ \text{Verify}(\dots, m^*, \text{ADM}^*, \sigma^*) = 1 \\ \wedge (m^*, \text{ADM}^*, \sigma^*) \notin Q \end{array} \right] = \text{negl}(\lambda).$$

Definition 4 (Immutability [6]). UOV-San is immutable if for every PPT adversary $\mathcal{A}$ holding (sk_{San}, tk) but not sk_S :

$$\text{Adv}_{\text{UOV-San}, \mathcal{A}}^{\text{IMM}}(\lambda) := \Pr \left[\begin{array}{l} (m^*, \text{ADM}^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sign}}}(pk_S, pk_{\text{San}}, sk_{\text{San}}, tk) : \\ \text{Verify} = 1 \wedge \exists i \notin \text{ADM}^* : m_i^* \neq m_i \end{array} \right] = \text{negl}(\lambda).$$

Definition 5 (Transparency [6]). UOV-San is transparent if for every PPT distinguisher $\mathcal{D}$, every $m, \text{ADM}, \text{MOD} \subseteq \text{ADM}$, and every honestly generated σ on (m, ADM) :

$$\text{Adv}_{\text{UOV-San}, \mathcal{D}}^{\text{TRANS}}(\lambda) := |\Pr[\mathcal{D}(\sigma, \sigma') = 1] - \Pr[\mathcal{D}(\sigma, \sigma_{\text{fresh}}) = 1]| = \text{negl}(\lambda),$$

where $\sigma' \leftarrow \text{Sanitize}(\dots, \sigma, \text{MOD})$ and σ_{fresh} is a fresh signature on the sanitised message m' . (UOV-San does not achieve this property; see Section 7.)

Definition 6 (Accountability). UOV-San is accountable if no PPT adversary can falsely attribute the origin of a valid signature version to the other party. We consider two games.

Signer-framing game. The adversary is given sk_S but not sk_{San} . It may interact with the signing and sanitizing oracles and finally outputs $(m^*, \text{ADM}^*, \sigma^*)$. It wins if

$$\text{Verify}(pk_S, pk_{\text{San}}, ck, m^*, \text{ADM}^*, \sigma^*) = 1$$

and

$$\text{Judge}(pk_S, pk_{\text{San}}, m^*, \text{ADM}^*, \sigma^*) = \text{Sanitizer},$$

but σ^* was not output by the honest sanitizing oracle.

Table 1: UOV parameter sets [27]. PK sizes include the seed-expanded representation; Sig is the raw signature size in bytes. Security levels follow NIST categories.

Level	q	o	v	n	PK (KB)	Sig (B)
I	2^8	44	72	116	66.6	116
III	2^8	72	118	190	278.4	190
V	2^8	96	148	244	838.0	244

Sanitizer-framing game. The adversary is given (sk_{San}, tk) but not sk_S . It may interact with the signing oracle and finally outputs (m^*, ADM^*, σ^*) . It wins if

$$\text{Verify}(pk_S, pk_{San}, ck, m^*, ADM^*, \sigma^*) = 1$$

and

$$\text{Judge}(pk_S, pk_{San}, m^*, ADM^*, \sigma^*) = \text{Signer},$$

but σ^* was not output by the honest signing protocol.

The advantage $\text{Adv}_{\text{UOV-San}, \mathcal{A}}^{\text{ACC}}(\lambda)$ is the maximum of the winning probabilities in the two games. The scheme is accountable if this advantage is negligible for every PPT adversary $\mathcal{A}$.

4 The Unbalanced Oil-and-Vinegar Scheme

UOV was proposed by Kipnis, Patarin, and Goubin [15] as a repair of Patarin’s Oil-and-Vinegar scheme [21] (which was broken by Kipnis and Shamir [16]). UOV has withstood over 25 years of cryptanalysis and was submitted to NIST’s additional signatures process [27].

4.1 Parameters and Key Generation

UOV is parameterised by a triple (q, o, v) where $q = 2^k$ is the field size, o is the number of *oil* variables, and v is the number of *vinegar* variables, with $n = o + v$ and $v \geq o$. Table 1 summarises the NIST submission parameters [27].

The secret key consists of:

- A linear bijection $T : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ (the oil-vinegar transformation), represented as an $n \times n$ invertible matrix over $\mathbb{F}_q$.
- A set of o quadratic polynomials $\mathbf{F} = (f_1, \dots, f_o) : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^o$ in the *central map* form, where each f_j has the special structure that its oil-oil terms vanish:

$$f_j(\mathbf{x}) = \sum_{k=1}^v \sum_{\ell=1}^n \alpha_{j,k,\ell} x_k x_\ell + \sum_{k=1}^n \beta_{j,k} x_k + \gamma_j,$$

i.e., the coefficients of $x_i x_j$ for $i, j > v$ are zero.

The public key is $P = \mathbf{F} \circ T : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^o$, a quadratic map with no apparent structure.

4.2 Signing and Verification

Signing. To sign a digest $h \in \mathbb{F}_q^o$:

1. Sample a vinegar vector $\mathbf{a} \xleftarrow{\$} \mathbb{F}_q^v$.
2. Compute the $o \times o$ matrix $M_{\mathbf{a}}$ and vector $\mathbf{c}_{\mathbf{a}}$ by substituting $\mathbf{a}$ into the vinegar positions of $\mathbf{F}$.
3. If $M_{\mathbf{a}}$ is singular (probability $p_{\text{sing}} \approx q^{-1}$), resample $\mathbf{a}$.
4. Solve the linear system $M_{\mathbf{a}} \cdot \mathbf{x}_o = h - \mathbf{c}_{\mathbf{a}}$ for the oil vector $\mathbf{x}_o \in \mathbb{F}_q^o$.
5. Output $\sigma := T^{-1}(\mathbf{a} \parallel \mathbf{x}_o) \in \mathbb{F}_q^n$.

Expected number of iterations: $1/(1 - p_{\text{sing}}) \approx 256/255 \approx 1.004$. Each iteration costs $O(o^3)$ for the linear solve plus $O(no^2)$ for computing $M_{\mathbf{a}}$.

Verification. Given (σ, h) : accept iff $P(\sigma) = h$.

4.3 Security Assumptions

Definition 7 (MQ Hardness). *The MQ problem is hard: for every PPT adversary $\mathcal{A}$,*

$$\Pr[P(\mathbf{x}) = \mathbf{y} \mid \mathbf{x} \leftarrow \mathcal{A}(P, \mathbf{y})] = \text{negl}(\lambda),$$

where $P \xleftarrow{\$} \text{MQ}(q, o, n)$ and $\mathbf{y} \xleftarrow{\$} \mathbb{F}_q^o$.

Definition 8 (OVD [15]). *The Oil-Vinegar Distinguishing assumption states that no PPT algorithm distinguishes a UOV public key $P = \mathbf{F} \circ T$ from a uniformly random MQ map with non-negligible advantage:*

$$\text{Adv}_{\mathcal{D}}^{\text{OVD}}(\lambda) := \left| \Pr[\mathcal{D}(P) = 1 \mid P = \mathbf{F} \circ T] - \Pr[\mathcal{D}(P) = 1 \mid P \xleftarrow{\$} \text{MQ}(q, o, n)] \right| = \text{negl}(\lambda).$$

The best known attack against OVD is the “rectangular MinRank” attack of Beullens [4], requiring $\tilde{O}(q^{v-o})$ operations. For $v \geq 2o$, this is at least $\tilde{O}(q^o)$, matching the MQ hardness bound.

Remark 1 (Singularity probability and output distribution). The probability $p_{\text{sing}} = O(q^{-1}) \approx 1/256$ that $M_{\mathbf{a}}$ is singular is a *positive constant* for fixed $q = 2^8$, not a negligible function of λ . Consequently, the output distribution of UOV signing is *not* statistically uniform over $\mathbb{F}_q^n$, but only computationally indistinguishable from uniform under OVD. This distinction is critical for understanding why transparency is structurally unachievable in our construction (Section 7).

5 Target Second-Preimage Resistance and MV-CH

This section formalises the target second-preimage resistance assumption used by UOV-San and defines MV-CH as a dedicated target-collision trapdoor hash. We intentionally avoid claiming that MV-CH is a standard message-dependent chameleon hash. MV-CH hashes only the randomness r , while the message content is bound by the Layer 2 signature in Section 6.

5.1 Target Second-Preimage Resistance for UOV

Let

$$P : \mathbb{F}_q^n \longrightarrow \mathbb{F}_q^o$$

be a UOV public map, where $n = o + v$. Since $n > o$, the map is underdetermined and a typical image point has a large fibre of expected size about $q^{n-o} = q^v$. Therefore, for a random input r , many distinct $r' \neq r$ satisfying $P(r') = P(r)$ are expected to exist. The security question is not whether such an r' exists, but whether it can be found efficiently without the UOV trapdoor.

Definition 9 (tSPR for UOV). Let $\text{UOV.KeyGen}(1^\lambda)$ generate a key pair (P, sk) , where $P : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^o$ is a UOV public map. The target second-preimage resistance assumption for UOV states that for every efficient adversary $\mathcal{A}$,

$$\text{Adv}_{\text{UOV}, \mathcal{A}}^{\text{tSPR}}(\lambda) := \Pr \left[\begin{array}{l} (P, sk) \leftarrow \text{UOV.KeyGen}(1^\lambda); \\ r \xleftarrow{\$} \mathbb{F}_q^n; \\ r' \leftarrow \mathcal{A}(P, r) : \\ r' \neq r \wedge P(r') = P(r) \end{array} \right]$$

is negligible.

The adversary receives the target input r after the public key has been generated. This differs from arbitrary collision resistance, where the adversary may choose both r and r' . For a quadratic map, arbitrary collisions are easy: fixing a nonzero difference Δ , the equation

$$P(x + \Delta) - P(x) = 0$$

is affine linear in x , and therefore can often be solved efficiently for x . This does not violate tSPR, because in the tSPR game the input r is fixed by the challenger and the adversary must solve

$$P(r + \Delta) - P(r) = 0$$

for a nonzero Δ . This is a known-solution MQ system rather than an arbitrary-collision search.

Remark 2 (Relation to UOV strong unforgeability). Standard EUF-CMA security of hash-and-sign UOV concerns the hardness of finding a valid preimage for a fresh digest. Strong unforgeability also requires that, given one valid signature σ on a digest $h = P(\sigma)$, it is infeasible to find another valid signature $\sigma' \neq \sigma$ with $P(\sigma') = h$. This latter task is exactly a target second-preimage problem for the UOV public map. Thus, the strong-unforgeability assumption used in this paper decomposes into fresh-target UOV unforgeability plus tSPR for same-target re-signing.

5.2 Post-Quantum Cryptanalytic Analysis of tSPR

We now analyse tSPR against generic, algebraic, structural, and quantum attacks. The discussion is not a proof of tSPR from standard MQ hardness. Rather, it identifies the relevant attack surfaces and gives the conservative security interpretation used for the parameter choices.

Direct generic search. Given r and $y = P(r)$, a uniformly random trial $x \in \mathbb{F}_q^n$ satisfies $P(x) = y$ with probability approximately q^{-o} , assuming that P behaves like a random map onto $\mathbb{F}_q^o$. Hence a direct classical random-search attack costs about

$$q^o$$

evaluations of P . Since the trivial solution $x = r$ occurs with probability q^{-n} , it does not materially affect the estimate. A quantum adversary can apply amplitude amplification to the predicate $P(x) = P(r)$, obtaining a direct quantum search cost

$$q^{o/2}.$$

For the parameters of Table 1 with $q = 2^8$, this gives the following direct-search estimates:

Level	o	classical direct search	quantum direct search
I	44	2^{352}	2^{176}
III	72	2^{576}	2^{288}
V	96	2^{768}	2^{384}

These numbers should be interpreted only as generic second-preimage costs. They are not the final claimed security levels, because structural attacks against UOV may be cheaper.

Differential arbitrary-collision attacks. For any fixed nonzero Δ , the expression

$$P(x + \Delta) - P(x)$$

is affine linear in x . Thus an attacker can efficiently search for arbitrary collisions by choosing Δ and solving a linear system for x . This attack is irrelevant to tSPR because it lets the attacker choose the target x . In UOV-San, the target randomness r_i is sampled by the signer and then bound into the signature through $H_2(P_{\text{CH}}(r_i))$. An attacker trying to alter an already signed object must collide with this fixed r_i , not with a chosen input.

For a fixed target r , writing $r' = r + \Delta$ gives

$$P(r + \Delta) - P(r) = 0.$$

This is a system of o quadratic equations in the n unknowns Δ , with the known trivial solution $\Delta = 0$. The attacker must find a nonzero solution. The existence of the trivial solution does not linearise the system; it only shows that the system is not a random inhomogeneous MQ instance. Known-solution MQ systems of this form remain the relevant direct algebraic formulation of tSPR.

Direct algebraic attacks. A direct algebraic attacker may solve

$$P(x) = P(r)$$

or equivalently

$$P(r + \Delta) - P(r) = 0, \quad \Delta \neq 0.$$

The system is underdetermined because $n = o + v > o$, and the expected number of solutions is large. This fact alone does not imply an efficient algorithm for finding a second preimage under a hidden-structure UOV public map. Hybrid Gröbner-basis or XL-type algorithms can reduce the number of variables by guessing a subset of variables and solving the remaining system. The relevant complexity depends on the degree of regularity, the field size, and the number of guessed variables. We do not claim that the direct algebraic complexity is exactly q^o . Instead, our conservative position is that direct algebraic attacks are part of the standard UOV cryptanalytic surface. If such an attack solved the same-target preimage problem substantially faster than the best known UOV attacks, it would also endanger strong unforgeability of UOV signatures, since it would transform one valid signature on a digest into another valid signature on the same digest.

Structural UOV attacks. An attacker may attempt to recover the hidden oil-vinegar structure of the public map $P = F \circ T$. Once the trapdoor or an equivalent oil-vinegar structure is recovered, tSPR is easy: the attacker can run the UOV signing algorithm on the target $P(r)$ and obtain another preimage. Therefore tSPR security is upper-bounded by the cost of UOV key recovery.

The relevant structural attacks include Kipnis-Shamir type attacks, rank and MinRank attacks, and their refinements, including rectangular MinRank attacks. The parameters used in this paper are the UOV submission parameters listed in Table 1. Consequently, the tSPR security claim must be interpreted as

$$\min\{\text{direct second-preimage cost, UOV structural key-recovery cost}\}.$$

Since the direct generic costs above are far above the NIST target levels, the conservative security level is governed by the UOV parameter category rather than by q^o .

Quantum attacks. There are three relevant quantum attack classes.

[leftmargin=*]

1. *Direct amplitude amplification.* Searching for x such that $P(x) = P(r)$ gives the generic cost $q^{o/2}$, as discussed above.
2. *Quantum acceleration of structural attacks.* Any classical UOV key-recovery attack with a dominant exhaustive search component may receive a square-root speed-up from Grover-type techniques. Therefore the post-quantum security of tSPR cannot exceed the post-quantum security of the underlying UOV parameter set.

Table 2: Conservative interpretation of tSPR attack costs. Direct search costs are much larger than the target categories; the effective claim is limited by UOV structural security and the random-oracle output length.

Level	o	Direct classical	Direct quantum	Conservative claim
I	44	2^{352}	2^{176}	$\min(\text{UOV Cat. I}, 2^{\kappa/2})$
III	72	2^{576}	2^{288}	$\min(\text{UOV Cat. III}, 2^{\kappa/2})$
V	96	2^{768}	2^{384}	$\min(\text{UOV Cat. V}, 2^{\kappa/2})$

3. *Quantum random-oracle attacks.* MV-CH outputs $H_2(P(r))$. A target collision caused only by H_2 , namely

$$H_2(P(r')) = H_2(P(r)) \quad \text{with} \quad P(r') \neq P(r),$$

is a target second-preimage attack on a κ -bit random oracle. In the quantum setting, amplitude amplification gives cost about $2^{\kappa/2}$. With $\kappa = 256$, this is 2^{128} , matching NIST Level I target security. Higher security levels should increase κ accordingly if a full QROM-level margin is desired.

Conservative security interpretation. The effective post-quantum security of the MV-CH tSPR layer is therefore bounded by

$$\min \left\{ q^{o/2}, 2^{\kappa/2}, \text{post-quantum cost of UOV structural key recovery} \right\}.$$

For $\kappa = 256$, the random-oracle target-collision bound is 2^{128} . For Levels III and V, one may either keep $\kappa = 256$ and obtain a Level-I hash-output bottleneck, or increase κ to 384 and 512 bits, respectively, to align the random-oracle target second-preimage cost with NIST Categories III and V.

Remark 3 (What tSPR does and does not claim). The tSPR assumption does not claim arbitrary collision resistance of the UOV public map. It also does not claim a reduction from standard MQ inversion. It is a same-target second-preimage assumption for UOV public maps and is exactly the property needed when an adversary attempts to change the randomness embedded in an already valid UOV-San signature.

5.3 MV-CH as a Target-Collision Trapdoor Hash

We now instantiate the TCTH syntax of Section 2.4 using an independent UOV key pair. The construction is denoted MV-CH.

TCTH.KeyGen(1^λ). Run

$$(P_{\text{CH}}, (T_{\text{CH}}, F_{\text{CH}})) \leftarrow \text{UOV.KeyGen}(1^\lambda).$$

Set

$$ck := P_{\text{CH}}, \quad tk := (T_{\text{CH}}, F_{\text{CH}}).$$

Output (ck, tk) . The key pair (ck, tk) is independent of both the signer's UOV key pair (pk_S, sk_S) and the sanitizer's UOV signing key pair $(pk_{\text{San}}, sk_{\text{San}})$.

TCTH.Hash(ck, r). Given $r \in \mathbb{F}_q^n$, compute

$$y := P_{\text{CH}}(r) \in \mathbb{F}_q^o$$

and output

$$c := H_2(y) = H_2(P_{\text{CH}}(r)).$$

The hash takes no message argument. Message binding is achieved at the UOV-San level through the Layer 2 digest

$$h_{\text{San}} = \text{HashToF}(H_{\text{San}}(m_{\overline{\text{ADM}}}, (m_i, r_i)_{i \in \text{ADM}}, \text{ADM}, pk_S)).$$

TCTH.TrapColl(tk, r). Given $tk = (T_{\text{CH}}, F_{\text{CH}})$ and $r \in \mathbb{F}_q^n$.
[leftmargin=*]

1. Compute $y^* := P_{\text{CH}}(r)$.
2. Run the UOV signing procedure with secret key tk and target y^* . That is, sample a vinegar vector $a \xleftarrow{\$} \mathbb{F}_q^v$, form the corresponding linear system

$$M_a x_o = y^* - c_a,$$

resample a if M_a is singular, solve for x_o , and output

$$r' := T_{\text{CH}}^{-1}(a \| x_o).$$

3. If $r' = r$, repeat the procedure. This rejection has expected constant overhead.

By UOV correctness,

$$P_{\text{CH}}(r') = y^* = P_{\text{CH}}(r),$$

and therefore

$$\text{TCTH.Hash}(ck, r') = H_2(P_{\text{CH}}(r')) = H_2(P_{\text{CH}}(r)) = \text{TCTH.Hash}(ck, r).$$

Theorem 2 (Target second-preimage resistance of MV-CH). *Assume the tSPR assumption for the UOV public map P_{CH} . In the classical random-oracle model for H_2 , for every efficient adversary $\mathcal{A}$ that outputs $r' \neq r$ with*

$$\text{TCTH.Hash}(ck, r') = \text{TCTH.Hash}(ck, r),$$

there exists an efficient adversary $\mathcal{B}$ against UOV tSPR such that

$$\text{TSPR}_{\text{MV-CH}, \mathcal{A}}(\lambda) \leq \text{Adv}_{\text{UOV}, \mathcal{B}}^{\text{tSPR}}(\lambda) + \frac{q_{H_2}^2}{2^{\kappa+1}}.$$

Proof. The reduction $\mathcal{B}$ receives a UOV tSPR challenge (P, r^*) , where $r^* \xleftarrow{\$} \mathbb{F}_q^n$. It sets $ck := P$ and gives (ck, r^*) to $\mathcal{A}$. The random oracle H_2 is simulated by lazy sampling. Suppose $\mathcal{A}$ outputs $r' \neq r^*$ such that $H_2(P(r')) = H_2(P(r^*))$. There are two cases. If $P(r') = P(r^*)$, then r' is a valid solution to the UOV tSPR challenge, and $\mathcal{B}$ outputs r' . Otherwise, $P(r') \neq P(r^*)$ but the two distinct inputs collide under the random oracle H_2 . In the classical random-oracle model, the probability of such a collision among at most q_{H_2} oracle queries is bounded by $q_{H_2}^2/2^{\kappa+1}$. Combining the two cases gives the claimed bound.

Remark 4 (Quantum random-oracle caveat). The theorem above is stated in the classical random-oracle model, as are the main security proofs of this paper. Against quantum random-oracle queries, the H_2 -only target-collision term should be interpreted heuristically as a target second-preimage cost of about $2^{\kappa/2}$ under amplitude amplification. A full QROM proof is left as future work.

Theorem 3 (Trapdoor collision correctness). *For honestly generated MV-CH keys and every $r \in \mathbb{F}_q^n$, $\text{TCTH.TrapColl}(tk, r)$ runs in expected polynomial time and outputs $r' \neq r$ satisfying*

$$P_{\text{CH}}(r') = P_{\text{CH}}(r),$$

and hence

$$\text{TCTH.Hash}(ck, r') = \text{TCTH.Hash}(ck, r).$$

The expected number of vinegar resampling trials is

$$\frac{1}{1 - p_{\text{sing}}},$$

where $p_{\text{sing}} \approx q^{-1}$ is the probability that the oil linear system is singular.

Proof. The algorithm sets $y^* = P_{\text{CH}}(r)$ and then runs the UOV signing algorithm with target y^* . By UOV correctness, the output r' satisfies $P_{\text{CH}}(r') = y^* = P_{\text{CH}}(r)$. Therefore the hash values are equal.

The probability that the UOV signing procedure returns exactly the input r is at most the probability of sampling the same internal vinegar component that corresponds to $T_{\text{CH}}(r)$, namely about q^{-v} , up to the conditioning introduced by singular-matrix rejection. If this rare event occurs, the algorithm simply repeats. Thus the expected overhead of enforcing $r' \neq r$ is negligible, and the dominant expected cost remains that of one UOV signing operation.

The output distribution is the UOV signing distribution conditioned on the target $P_{\text{CH}}(r)$. We do not require, and do not claim, that this distribution is statistically uniform over the full fibre $P_{\text{CH}}^{-1}(P_{\text{CH}}(r))$. In particular, because the singular-matrix rejection probability is about $1/q$ for fixed $q = 2^8$, the induced distribution may have a non-negligible statistical bias. This bias is irrelevant for correctness and for the tSPR-based security argument used in UOV-San.

6 The UOV-San Construction

6.1 Design Rationale

The UOV-San construction is driven by three design principles, each addressing one of the challenges in Section 1.3.

Principle 1: Dual-signature architecture. We use two independent UOV signatures:

- **Layer 1 signature** σ_S (signer, using sk_S): covers the fixed blocks $m_{\overline{\text{ADM}}}$ and the MV-CH commitments $(c_i)_{i \in \text{ADM}}$ where $c_i := H_2(P_{CH}(r_i))$. The commitments are invariant under sanitization because TCTH.TrapColl produces r'_i with $P_{CH}(r'_i) = P_{CH}(r_i)$, hence $c'_i = c_i$. Thus σ_S is preserved unchanged across all sanitizations.
- **Layer 2 signature** σ_{San} (sanitizer, using sk_{San}): covers the fixed blocks $m_{\overline{\text{ADM}}}$, the full admissible block content $(m_i, r_i)_{i \in \text{ADM}}$, and the signer's public key pk_S . This signature is recomputed after each sanitization, binding the current message content.

This resolves Challenge 1: σ_S is preserved (correctness) while σ_{San} binds the admissible blocks (immutability). Table 3 summarises the coverage and role of each signature component.

Principle 2: Implementable target-collision trapdoor hash. MV-CH is used as a specialised target-collision trapdoor hash:

$$\text{TCTH.Hash}(ck, r) := H_2(P_{CH}(r)).$$

It depends only on the randomness r . Thus $\text{TCTH.TrapColl}(tk, r)$ finds r' with

$$\text{TCTH.Hash}(ck, r') = \text{TCTH.Hash}(ck, r)$$

by running UOV signing with target $P_{CH}(r)$. No random-oracle programming is required. Message binding is provided by the Layer 2 signature, not by MV-CH itself.

Principle 3: Interactive signing for key separation. The initial signing is a two-message protocol where the signer produces σ_S using only sk_S , and the sanitizer produces σ_{San} using only sk_{San} . Neither party ever holds the other's private key. The sanitizer's verification of σ_S in Sign_2 (Algorithm 3) is essential: without it, a malicious signer could send an invalid σ_S and later claim the sanitizer endorsed an illegitimate message. This resolves Challenge 3.

Key architecture. The scheme uses three independent UOV key pairs:

- (pk_S, sk_S) : signer's signing key pair.
- (pk_{San}, sk_{San}) : sanitizer's signing key pair (for Layer 2 signatures and proofs).
- $(ck, tk) = (pk_{CH}, sk_{CH})$: MV-CH target-collision trapdoor-hash key pair, used for commitments and trapdoor collisions.

This strict mathematical separation is essential for accountability. Although the sEUF-CMA of the signing keys and the target second-preimage resistance of the MV-CH both fundamentally rely on the tSPR hardness of UOV, they operate on independent public maps. Therefore, a tSPR break or authorized trapdoor usage in the chameleon hash (ck) yields no signing capability under pk_{San} or pk_S , and a signing forgery does not directly compromise the chameleon hash.

Table 3: Dual-signature architecture: coverage and security role of each component.

Component Key		Covers	Enforces
σ_S	sk_S only	$m_{\overline{ADM}}, (c_i)_{i \in ADM}, ADM, pk_{San}$	Unforgeability, Immutability
σ_{San}	sk_{San} only	$m_{\overline{ADM}}, (m_i, r_i)_{i \in ADM}, ADM, pk_S$	Immutability, Accountability
$(r_i)_{i \in ADM}$	—	Chameleon randomness	Correctness of σ_S reuse
(ω, τ)	sk_S or sk_{San}	entire signature state	Accountability (origin binding)

6.2 Signature Structure

The signature is $\sigma = (\sigma_S, \sigma_{San}, (r_i)_{i \in ADM}, \omega, \tau)$, where:

$$\begin{aligned}
\sigma_S &:= \text{UOV.Sign}(sk_S, h_S), \\
h_S &:= \text{HashToF}(H_S(m_{\overline{ADM}}, (c_i)_{i \in ADM}, ADM, pk_{San})), \quad c_i := H_2(P_{CH}(r_i)), \\
\sigma_{San} &:= \text{UOV.Sign}(sk_{San}, h_{San}), \\
h_{San} &:= \text{HashToF}(H_{San}(m_{\overline{ADM}}, (m_i, r_i)_{i \in ADM}, ADM, pk_S)), \\
\omega &\in \{\text{Signer}, \text{Sanitizer}\} \text{ is the origin tag of the current signature version,} \\
\tau &:= \begin{cases} \text{UOV.Sign}(sk_S, \text{HashToF}(H_J(m, ADM, \sigma_S, \sigma_{San}, (r_i)_{i \in ADM}, \text{Signer}))) & \text{if } \omega = \text{Signer}, \\ \text{UOV.Sign}(sk_{San}, \text{HashToF}(H_J(m, ADM, \sigma_S, \sigma_{San}, (r_i)_{i \in ADM}, \text{Sanitizer}))) & \text{if } \omega = \text{Sanitizer}. \end{cases}
\end{aligned}$$

The inclusion of pk_{San} in h_S and pk_S in h_{San} prevents cross-key replay attacks. All hash inputs use the length-prefixed encoding of Section 2.

Accountability tag. Each valid signature additionally contains an accountability tag (ω, τ) , where $\omega \in \{\text{Signer}, \text{Sanitizer}\}$ indicates the origin of the current version of the signature. The tag is generated at the time the corresponding version is produced and is embedded into the signature; it is not generated afterwards. The tag binds the claimed origin to the complete current signature tuple and message. Hence a tag produced for one version cannot be reused for another version except with negligible probability under collision resistance of H_J .

6.3 Algorithms

We present all eight algorithms of UOV-San. Each algorithm is referenced in the security proofs of Section 8 and the correctness analysis of Proposition 1.

Algorithm 1 (KeyGen) generates three independent UOV key pairs and distributes them according to the key separation invariant. The signer never receives sk_{San} or tk .

Algorithm 1 UOV-San.KeyGen(1^λ)

-
- 1: $(pk_S, sk_S) \leftarrow \text{UOV.KeyGen}(1^\lambda)$ $\triangleright$ signer's signing key pair
 - 2: $(pk_{San}, sk_{San}) \leftarrow \text{UOV.KeyGen}(1^\lambda)$ $\triangleright$ sanitizer's signing key pair; independent of (pk_S, sk_S)
 - 3: $(ck, tk) \leftarrow \text{TCTH.KeyGen}(1^\lambda)$ $\triangleright ck = P_{CH}, tk = (T_{CH}, \mathbf{F}_{CH})$; independent of both above
 - 4: **Signer receives:** $(sk_S, pk_S, pk_{San}, ck)$ $\triangleright$ signer does NOT receive sk_{San} or tk
 - 5: **Sanitizer receives:** $(sk_{San}, tk, pk_{San}, pk_S, ck)$ $\triangleright$ sanitizer does NOT receive sk_S
 - 6: **Public parameters:** (pk_S, pk_{San}, ck)
-

Algorithm 2 (Sign_1) is the signer's first message in the interactive protocol. The signer samples fresh randomness r_i for each admissible block, computes MV-CH commitments $c_i = H_2(P_{CH}(r_i))$, and signs the Layer 1 digest using only sk_S . The commitments bind the randomness without revealing the message content.

Algorithm 2 $\text{Sign}_1(sk_S, pk_{San}, ck, m, \text{ADM})$ [Signer's first message; uses only sk_S and public ck]

-
- 1: **for** each $i \in \text{ADM}$ **do**
 - 2: $r_i \xleftarrow{\$} \mathbb{F}_q^n$ $\triangleright$ fresh randomness for each admissible block
 - 3: $c_i \leftarrow H_2(P_{CH}(r_i))$ $\triangleright$ MV-CH commitment: binds r_i without revealing m_i
 - 4: **end for**
 - 5: $h_S \leftarrow \text{HashToF}(H_S(m_{\overline{\text{ADM}}}, (c_i)_{i \in \text{ADM}}, \text{ADM}, pk_{San}))$ $\triangleright$ Layer 1 digest: covers fixed blocks and commitments only
 - 6: $\sigma_S \leftarrow \text{UOV.Sign}(sk_S, h_S)$ $\triangleright$ Layer 1 signature: reused unchanged in all sanitizations
 - 7: $\text{msg}_1 \leftarrow (m, \text{ADM}, \sigma_S, (r_i)_{i \in \text{ADM}}, (c_i)_{i \in \text{ADM}})$ $\triangleright$ send to sanitizer for verification and co-signing
 - 8: $\text{st} \leftarrow (m, \text{ADM}, \sigma_S, (r_i)_{i \in \text{ADM}})$ $\triangleright$ signer retains state for Sign_3
 - 9: **return** $(\text{msg}_1, \text{st})$
-

Algorithm 3 (Sign_2) is the sanitizer's response. The sanitizer verifies the consistency of the signer's commitment and the validity of σ_S before producing σ_{San} . This verification is essential for accountability: it ensures the sanitizer only co-signs messages the signer has legitimately authorised. Note that Sign_2 does not use tk ; trapdoor collisions are only needed in **Sanitize**.

Algorithm 3 $\text{Sign}_2(sk_{San}, pk_S, pk_{San}, ck, \text{msg}_1)$ [Sanitizer's response; uses only sk_{San} ; does not use tk]

```

1: Parse  $\text{msg}_1 = (m, \text{ADM}, \sigma_S, (r_i)_{i \in \text{ADM}}, (c_i)_{i \in \text{ADM}})$ 
2: for each  $i \in \text{ADM}$  do
3:   if  $H_2(P_{CH}(r_i)) \neq c_i$  then
4:     return  $\perp$  ▷ reject: signer's commitment inconsistent with  $r_i$ 
5:   end if
6: end for
7:  $h_S \leftarrow \text{HashToF}(H_S(m_{\overline{\text{ADM}}}, (c_i)_{i \in \text{ADM}}, \text{ADM}, pk_{San}))$  ▷ recompute Layer 1 digest from received data
8: if  $\text{UOV.Verify}(pk_S, h_S, \sigma_S) \neq 1$  then
9:   return  $\perp$  ▷ reject: signer's Layer 1 signature invalid
10: end if
11:  $h_{San} \leftarrow \text{HashToF}(H_{San}(m_{\overline{\text{ADM}}}, (m_i, r_i)_{i \in \text{ADM}}, \text{ADM}, pk_S))$  ▷ Layer 2 digest: covers full message content
12:  $\sigma_{San} \leftarrow \text{UOV.Sign}(sk_{San}, h_{San})$  ▷ Layer 2 signature: recomputed at each sanitization
13: return  $\text{msg}_2 := \sigma_{San}$ 

```

Algorithm 4 (Sign_3) assembles the final signature and creates the signer-origin accountability tag. If the sanitizer rejected ($\sigma_{San} = \perp$), the signer aborts. Otherwise, the dual signature together with the tag is assembled.

Algorithm 4 $\text{Sign}_3(sk_S, \text{st}, \text{msg}_2)$ [Signer assembles final signature and creates origin tag]

```

1: Parse  $\text{st} = (m, \text{ADM}, \sigma_S, (r_i)_{i \in \text{ADM}})$ 
2: Parse  $\text{msg}_2 = \sigma_{San}$ 
3: if  $\sigma_{San} = \perp$  then
4:   return  $\perp$  ▷ sanitizer rejected; abort signing
5: end if
6:  $\omega \leftarrow \text{Signer}$ 
7:  $d_{\text{Signer}} \leftarrow \text{HashToF}(H_J(m, \text{ADM}, \sigma_S, \sigma_{San}, (r_i)_{i \in \text{ADM}}, \omega))$ 
8:  $\tau \leftarrow \text{UOV.Sign}(sk_S, d_{\text{Signer}})$  ▷ origin tag for the initial version
9: return  $\sigma := (\sigma_S, \sigma_{San}, (r_i)_{i \in \text{ADM}}, \omega, \tau)$ 

```

Algorithm 5 (Verify) checks all three components: Layer 1 signature, Layer 2 signature, and the embedded accountability tag. All must be valid for acceptance.

Algorithm 5 $\text{UOV-San.Verify}(pk_S, pk_{San}, ck, m, \text{ADM}, \sigma)$ [Uses only public keys; checks signature layers and accountability tag]

```

1: Parse  $\sigma = (\sigma_S, \sigma_{San}, (r_i)_{i \in \text{ADM}}, \omega, \tau)$ 
2: for each  $i \in \text{ADM}$  do
3:    $c_i \leftarrow H_2(P_{CH}(r_i))$   $\triangleright$  recompute MV-CH commitments from randomness
4: end for
5:  $h_S \leftarrow \text{HashToF}(H_S(m_{\overline{\text{ADM}}}, (c_i)_{i \in \text{ADM}}, \text{ADM}, pk_{San}))$   $\triangleright$  Layer 1 digest
6: if  $\text{UOV.Verify}(pk_S, h_S, \sigma_S) \neq 1$  then
7:   return 0  $\triangleright$  Layer 1 check fails: signer's authorisation invalid
8: end if
9:  $h_{San} \leftarrow \text{HashToF}(H_{San}(m_{\overline{\text{ADM}}}, (m_i, r_i)_{i \in \text{ADM}}, \text{ADM}, pk_S))$   $\triangleright$  Layer 2 digest
10: if  $\text{UOV.Verify}(pk_{San}, h_{San}, \sigma_{San}) \neq 1$  then
11:   return 0  $\triangleright$  Layer 2 check fails: sanitizer's endorsement invalid
12: end if
13:  $d_\omega \leftarrow \text{HashToF}(H_J(m, \text{ADM}, \sigma_S, \sigma_{San}, (r_i)_{i \in \text{ADM}}, \omega))$   $\triangleright$  recompute digest for the
    accountability tag
14: if  $\omega = \text{Signer}$  then
15:   if  $\text{UOV.Verify}(pk_S, d_\omega, \tau) \neq 1$  then
16:     return 0
17:   end if
18: else if  $\omega = \text{Sanitizer}$  then
19:   if  $\text{UOV.Verify}(pk_{San}, d_\omega, \tau) \neq 1$  then
20:     return 0
21:   end if
22: else
23:   return 0
24: end if
25: return 1

```

Algorithm 6 (Sanitize) is the core sanitization operation. For each modified block $i \in I$, the sanitizer uses $\text{TCTH.TrapColl}(tk, r_i)$ to find new randomness r'_i satisfying $P_{CH}(r'_i) = P_{CH}(r_i)$, preserving the commitment c_i and hence σ_S . The Layer 2 signature is recomputed over the sanitized content, and a new sanitizer-origin accountability tag is created. Unchanged blocks retain their original randomness.

Algorithm 6 UOV-San.Sanitize($sk_{San}, tk, ck, pk_S, pk_{San}, m, ADM, \sigma, MOD$)
 [Sanitizer modifies admissible blocks; uses sk_{San} and tk]

```

1: Parse  $\sigma = (\sigma_S, \sigma_{San}, (r_i)_{i \in ADM}, \omega, \tau)$   $\triangleright$  old  $\omega, \tau$  are discarded
2: Parse  $MOD = \{(i, m'_i) : i \in I\}, I \subseteq ADM$   $\triangleright$  modification set: indices and new values
3: for each  $i \in ADM$  do
4:   if  $(i, m'_i) \in MOD$  then
5:      $r'_i \leftarrow \text{TCTH.TrapColl}(tk, r_i)$   $\triangleright$  find  $r'_i \neq r_i$  with  $P_{CH}(r'_i) = P_{CH}(r_i)$   $\triangleright$  ensures  $c'_i = H_2(P_{CH}(r'_i)) = c_i$ , preserving  $\sigma_S$ 
6:   else
7:      $r'_i \leftarrow r_i; m'_i \leftarrow m_i$   $\triangleright$  unchanged blocks: keep original randomness and content
8:   end if
9: end for
10:  $m' \leftarrow (m_{\overline{ADM}}, (m'_i)_{i \in ADM})$   $\triangleright m_{\overline{ADM}}$  is always unchanged
11:  $h'_{San} \leftarrow \text{HashToF}(H_{San}(m_{\overline{ADM}}, (m'_i, r'_i)_{i \in ADM}, ADM, pk_S))$   $\triangleright$  recompute Layer 2 digest for sanitized message
12:  $\sigma'_{San} \leftarrow \text{UOV.Sign}(sk_{San}, h'_{San})$   $\triangleright$  fresh Layer 2 signature on sanitized content
13:  $\omega' \leftarrow \text{Sanitizer}$ 
14:  $d_{\text{Sanitizer}} \leftarrow \text{HashToF}(H_J(m', ADM, \sigma_S, \sigma'_{San}, (r'_i)_{i \in ADM}, \omega'))$ 
15:  $\tau' \leftarrow \text{UOV.Sign}(sk_{San}, d_{\text{Sanitizer}})$   $\triangleright$  origin tag for the sanitized version
16: return  $(m', (\sigma_S, \sigma'_{San}, (r'_i)_{i \in ADM}, \omega', \tau'))$ 

```

Algorithm 7 (Proof) extracts the embedded accountability tag from the signature. Unlike previous versions, it does not create a new signature, thus preventing after-the-fact proof generation.

Algorithm 7 UOV-San.Proof(m, ADM, σ) [Extracts the embedded accountability tag]

```

1: Parse  $\sigma = (\sigma_S, \sigma_{San}, (r_i)_{i \in ADM}, \omega, \tau)$ 
2: return  $\pi := (\omega, \tau)$ 

```

Algorithm 8 (Judge) determines the origin of the current signature version by verifying the embedded accountability tag. It does not accept an external proof; the tag is already part of σ .

Algorithm 8 UOV-San.Judge($pk_S, pk_{San}, m, ADM, \sigma$) [Determines the origin of the current signature version]

```

1: Parse  $\sigma = (\sigma_S, \sigma_{San}, (r_i)_{i \in ADM}, \omega, \tau)$ 
2:  $d_\omega \leftarrow \text{HashToF}(H_J(m, ADM, \sigma_S, \sigma_{San}, (r_i)_{i \in ADM}, \omega))$ 
3: if  $\omega = \text{Signer}$  then
4:   if  $\text{UOV.Verify}(pk_S, d_\omega, \tau) = 1$  then
5:     return Signer
6:   else
7:     return  $\perp$ 
8:   end if
9: else if  $\omega = \text{Sanitizer}$  then
10:  if  $\text{UOV.Verify}(pk_{San}, d_\omega, \tau) = 1$  then
11:    return Sanitizer
12:  else
13:    return  $\perp$ 
14:  end if
15: else
16:  return  $\perp$ 
17: end if

```

6.4 Correctness

Proposition 1 (Correctness). *For all honestly generated keys, all m , ADM , $MOD \subseteq ADM$: (1) a signature produced by Algorithms 2–4 verifies under Algorithm 5; (2) a sanitised signature produced by Algorithm 6 verifies under Algorithm 5.*

Proof. **Part (1).** Sign_2 (Algorithm 3) verifies σ_S before proceeding, so σ_S is valid on h_S . σ_{San} is a fresh UOV signature on h_{San} , valid by UOV correctness. Sign_3 (Algorithm 4) sets $\omega = \text{Signer}$ and generates τ as a UOV signature under sk_S on d_{Signer} , which is correctly formed. Verify (Algorithm 5) recomputes h_S , h_{San} , and d_ω identically and accepts all three components.

Part (2). For each $i \in I$: $r'_i = \text{TCTH.TrapColl}(tk, r_i)$ satisfies $P_{CH}(r'_i) = P_{CH}(r_i)$ (Theorem 3), so $c'_i = H_2(P_{CH}(r'_i)) = H_2(P_{CH}(r_i)) = c_i$. Therefore:

$$h'_S = \text{HashToF}(H_S(m_{\overline{ADM}}, (c'_i)_{i \in ADM}, ADM, pk_{San})) = \text{HashToF}(H_S(m_{\overline{ADM}}, (c_i)_{i \in ADM}, ADM, pk_{San})) = h_S.$$

σ_S remains valid on $h'_S = h_S$. σ'_{San} is a fresh UOV signature on h'_{San} , valid by UOV correctness. Sanitize (Algorithm 6) sets $\omega' = \text{Sanitizer}$ and generates τ' as a UOV signature under sk_{San} on $d_{\text{Sanitizer}}$, which is correctly formed. Verify recomputes all digests and accepts. $\square$

7 Application Scope and Foregone Properties

UOV-San achieves unforgeability, immutability, and accountability, but does not achieve transparency or privacy. This section explains why these properties are foregone and precisely characterises the application scope.

7.1 Why Transparency and Privacy Are Foregone

Transparency. Transparency (Definition 5) requires that a verifier cannot distinguish a sanitised signature from a fresh signature on the same message. In UOV-San, the Layer 1 signature σ_S is reused unchanged across all sanitizations (see Table 3 and Algorithm 6). A verifier who holds both the original signature σ and the sanitised signature σ' can immediately detect that σ_S is identical in both, determining that sanitization occurred. This is a structural consequence of the dual-signature architecture: σ_S must be preserved to maintain the signer’s authorisation without requiring the signer’s participation in each sanitization, and this preservation makes σ_S a fingerprint of the original signing event.

Classical SSS constructions [2,6] achieve transparency by using a chameleon hash in the signer’s commitment layer such that σ_S itself changes across sanitizations via re-randomisation tied to the chameleon trapdoor. In our setting, this would require the sanitizer to hold a key related to sk_S , directly violating key separation and breaking accountability. Transparency and accountability are thus in fundamental tension in the dual-signature architecture.

Privacy. Privacy requires that admissible block content is hidden from verifiers. In UOV-San, the randomness r_i is included in the signature for verification (needed to recompute c_i in Algorithm 5). Given r_i and the public key pk_{San} , any verifier can compute $h_{San} = P_{San}(\sigma_{San})$ and enumerate candidates $\hat{m}$ to find m_i satisfying $H_{San}(m_{ADM}, \dots, (\hat{m}, r_i), \dots) = h_{San}$. Privacy is structurally unachievable without hiding r_i from verifiers, which would break verification.

7.2 Suitable Application Domains

The absence of transparency and privacy is operationally acceptable in applications where: (1) admissible block content is public or non-sensitive; (2) the fact of sanitization is not sensitive; (3) accountability is the primary requirement; (4) post-quantum security is required; and (5) fixed block integrity is required.

Supply chain manifests. A manufacturer signs a shipping manifest; a logistics provider sanitizes delivery details (route, timestamps). Accountability ensures disputes can be resolved; the sanitization event is not sensitive.

Government document redaction. A government agency signs a document; a disclosure officer sanitizes classified portions for public release. Redaction events are public by design; accountability ensures the officer cannot exceed their authorisation.

Blockchain audit trails. A transaction originator signs a record; an auditor sanitizes privacy-sensitive fields for regulatory reporting. Blockchain transparency means all parties can see the sanitized record.

Table 4: Security property comparison. “Unf.”=unforgeability, “Imm.”=immuability, “Trans.”=transparency, “Acc.”=accountability, “PQ”=post-quantum security, “×”=not achieved, “✓”=achieved.

Scheme	Unf.	Imm.	Trans.	Acc.	PQ
Ateniese et al. [2]	✓	✓	✓	✓	×
Brzuska et al. [6]	✓	✓	✓	✓	×
Lai et al. [19]	✓	✓	×	×	✓
Ahmad et al. [1]	✓	✓	✓	✓	✓
UOV-San (this work)	✓	✓	×	✓	✓

Scientific data publishing. A research institution signs a dataset; a data curator sanitizes participant identifiers for public release. Ethics approval typically requires disclosure of anonymisation procedures.

UOV-San is *not* suitable for applications where the fact of sanitization must be hidden, where admissible block content must be hidden from verifiers, or where the full Brzuska et al. five-property security is required.

7.3 Security Property Comparison

Table 4 compares UOV-San with prior SSS constructions. Our scheme is the only post-quantum SSS with full accountability and practical signature sizes.

8 Security Proofs

Let q_S and q_{San} denote the number of signing and sanitizing oracle queries, and let q_{H_S} , $q_{H_{San}}$, q_{H_2} , and q_{H_J} denote the number of queries to the corresponding random oracles. Let $a_{\max}$ be an upper bound on the number of admissible blocks in any message. We consider classical PPT adversaries in the random-oracle model.

We use the following bad events:

$$S, \quad San, \quad 2, \quad J,$$

where S denotes a collision among distinct inputs to H_S , San a collision among distinct inputs to H_{San} , 2 a collision among distinct inputs to H_2 , and J a collision among distinct inputs to H_J . By the birthday bound,

$$\Pr[S] \leq \frac{q_{H_S}^2}{2^{\kappa+1}}, \quad \Pr[San] \leq \frac{q_{H_{San}}^2}{2^{\kappa+1}},$$

$$\Pr[2] \leq \frac{q_{H_2}^2}{2^{\kappa+1}}, \quad \Pr[J] \leq \frac{q_{H_J}^2}{2^{\kappa+1}}.$$

All reductions below simulate the random oracles by lazy sampling.

UOV strong-forgery decomposition. Whenever an adversary outputs a valid UOV signature σ^* on a target h^* , there are two possibilities. If h^* was never signed before under the corresponding key, then (h^*, σ^*) is an EUF-CMA forgery. If h^* was signed before but σ^* is a new signature on the same target, then this is a target second-preimage attack on the corresponding UOV public map. Thus Assumption 1 is sufficient for the strong unforgeability claims used below.

8.1 Unforgeability

Theorem 4 (Unforgeability). *If UOV satisfies Assumption 1 and MV-CH is a target-collision trapdoor hash satisfying target second-preimage resistance, then UOV-San is unforgeable. More precisely, for every PPT adversary $\mathcal{A}$,*

$$\begin{aligned} \text{Adv}_{\text{UOV-San}, \mathcal{A}}^{\text{UNF}}(\lambda) &\leq 2\text{Adv}_{\text{UOV}}^{\text{sEUF-CMA}}(\lambda) + a_{\max} \text{Adv}_{\text{MV-CH}}^{\text{SPR}}(\lambda) \\ &\quad + \frac{q_{H_S}^2}{2^{\kappa+1}} + \frac{q_{H_{\text{San}}}^2}{2^{\kappa+1}} + \frac{q_{H_2}^2}{2^{\kappa+1}}. \end{aligned}$$

Proof. We use a game-hopping argument.

Game 0. This is the real unforgeability experiment. The adversary receives $(pk_S, pk_{\text{San}}, ck)$ and has access to $\mathcal{O}_{\text{Sign}}$ and $\mathcal{O}_{\text{San}}$. It wins if it outputs $(m^*, \text{ADM}^*, \sigma^*)$ such that $\text{Verify}(\dots) = 1$ and $(m^*, \text{ADM}^*, \sigma^*) \notin Q$.

Parse

$$\sigma^* = (\sigma_S^*, \sigma_{\text{San}}^*, (r_i^*)_{i \in \text{ADM}^*}, \omega^*, \tau^*).$$

For each $i \in \text{ADM}^*$ define $c_i^* := H_2(P_{CH}(r_i^*))$. Verification implies that $\text{UOV.Verify}(pk_S, h_S^*, \sigma_S^*) = 1$ and $\text{UOV.Verify}(pk_{\text{San}}, h_{\text{San}}^*, \sigma_{\text{San}}^*) = 1$, where

$$\begin{aligned} h_S^* &= \text{HashToF}(H_S(m_{\text{ADM}^*}^*, (c_i^*), \text{ADM}^*, pk_{\text{San}})), \\ h_{\text{San}}^* &= \text{HashToF}(H_{\text{San}}(m_{\text{ADM}^*}^*, (m_i^*, r_i^*), \text{ADM}^*, pk_S)). \end{aligned}$$

Game 1. Abort if $_S \vee_{\text{San}} \vee_2$ occurs. Hence

$$|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq \frac{q_{H_S}^2}{2^{\kappa+1}} + \frac{q_{H_{\text{San}}}^2}{2^{\kappa+1}} + \frac{q_{H_2}^2}{2^{\kappa+1}}.$$

We now condition on Game 1.

Signer-layer cases. If h_S^* was never signed under pk_S , then (h_S^*, σ_S^*) is an EUF-CMA forgery against the signer UOV instance. If h_S^* was signed before but σ_S^* is a new valid signature on the same target, then this gives a target second preimage for the signer UOV public map. Both are covered by the first UOV strong-forgery term.

If h_S^* and σ_S^* exactly match a previous oracle output, then since $_S$ does not occur, the input to H_S is identical. Hence the fixed blocks, the admissibility set, and all commitments (c_i) are the same as in that previous output.

If for some i we have $r_i^* \neq r_i$, then $H_2(P_{CH}(r_i^*)) = H_2(P_{CH}(r_i))$ and because $_2$ does not occur we get $P_{CH}(r_i^*) = P_{CH}(r_i)$, a target second-preimage attack against MV-CH. Guessing the index i loses a factor at most $a_{\max}$.

Sanitizer-layer cases. If the signer layer is not already explained, the novelty must be in the sanitizer layer or the tag. If h_{San}^* was never signed under pk_{San} , we obtain an EUF-CMA forgery; if it was signed before but σ_{San}^* is new, we obtain a target second preimage for the sanitizer UOV instance. These are covered by the second UOV strong-forgery term.

If both the signer and sanitizer layers are exactly reused from a previous oracle output, then the only possible novelty is the tag (ω^*, τ^*) . Since Verify accepts, τ^* is a valid UOV signature under either pk_S or pk_{San} on the digest $d_{\omega^*}^*$. If this tag signature is new, it is again a strong UOV forgery under the corresponding key. If it is not new, then the whole tuple coincides with a previous output, contradicting the forgery condition.

Therefore, in Game 1 every successful forgery yields either a strong UOV forgery under pk_S or pk_{San} , or an MV-CH target second-preimage attack. Adding the transition probability from Game 0 to Game 1 proves the theorem.

8.2 Immutability

Theorem 5 (Immutability). *If the signer UOV instance satisfies Assumption 1, then UOV-San is immutable. More precisely, for every PPT adversary $\mathcal{A}$ holding (sk_{San}, tk) but not sk_S ,*

$$\text{Adv}_{\text{UOV-San}, \mathcal{A}}^{\text{IMM}}(\lambda) \leq \text{Adv}_{\text{UOV}}^{\text{sEUF-CMA}}(\lambda) + \frac{q_{H_S}^2}{2^{\kappa+1}}.$$

Proof. Again we use two games.

Game 0. This is the real immutability experiment. The adversary knows (sk_{San}, tk) , has access to $\mathcal{O}_{\text{Sign}}$, and wins if it outputs a valid tuple $(m^*, \text{ADM}^*, \sigma^*)$ in which at least one fixed block has been changed, i.e., $\exists j \notin \text{ADM}^*$ with $m_j^* \neq m_j$.

Game 1. Abort if $_S$ occurs. Then

$$|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq \frac{q_{H_S}^2}{2^{\kappa+1}}.$$

Condition on Game 1. Parse $\sigma^* = (\sigma_S^*, \sigma_{San}^*, (r_i^*)_{i \in \text{ADM}^*}, \omega^*, \tau^*)$. Since verification accepts, $\text{UOV.Verify}(pk_S, h_S^*, \sigma_S^*) = 1$ with $h_S^* = \text{HashToF}(H_S(m_{\text{ADM}^*}^*, (c_i^*), \text{ADM}^*, pk_{San}))$.

Because a fixed block was changed, the input to H_S differs from the input used in the original signing query, except with event $_S$. Hence h_S^* is either a fresh target under pk_S , or it is an old target with a new valid signature. In the first case we obtain an EUF-CMA forgery; in the second case a target second preimage. Both are covered by Assumption 1.

The adversary's knowledge of sk_{San} and tk is irrelevant here, since the signer key pair is independent. Adding the probability of $_S$ gives the claimed bound.

8.3 Accountability

Theorem 6 (Accountability). *If UOV satisfies Assumption 1 and H_J is a random oracle, then UOV-San with embedded accountability tags is accountable. More precisely, for every PPT adversary $\mathcal{A}$,*

$$\text{Adv}_{\text{UOV-San}, \mathcal{A}}^{\text{ACC}}(\lambda) \leq 2\text{Adv}_{\text{UOV}}^{\text{sEUF-CMA}}(\lambda) + \frac{2q_{H_J}^2}{2^{\kappa+1}}.$$

Proof. We prove the two framing games separately.

Game A: malicious signer frames the sanitizer. The adversary knows sk_S but not sk_{San} . It wins only if it outputs a valid tuple for which $\text{Judge}(pk_S, pk_{San}, m^*, \text{ADM}^*, \sigma^*) = \text{Sanitizer}$, and the tuple was not output by the honest sanitizing oracle.

Parse $\sigma^* = (\sigma_S^*, \sigma_{San}^*, (r_i^*)_{i \in \text{ADM}^*}, \omega^*, \tau^*)$. Since the judge outputs **Sanitizer**, we have $\omega^* = \text{Sanitizer}$ and $\text{UOV.Verify}(pk_{San}, d_{\text{Sanitizer}}^*, \tau^*) = 1$ with $d_{\text{Sanitizer}}^* = \text{HashToF}(H_J(m^*, \text{ADM}^*, \sigma_S^*, \sigma_{San}^*, (r_i^*), \text{Sanitizer}))$.

Abort if J occurs; this costs at most $q_{H_J}^2/2^{\kappa+1}$. If $d_{\text{Sanitizer}}^*$ was never signed by the honest sanitizer, then τ^* is an EUF-CMA forgery under pk_{San} . If it was signed before but τ^* is a new valid signature on the same digest, this gives a target second preimage for the sanitizer UOV public map.

The only remaining case is that both the digest and the tag signature were previously produced by the honest sanitizer. Since J does not occur, the input to H_J must be identical, so the whole tuple coincides with a previous sanitizer output, contradicting the winning condition. Hence a malicious signer can frame the sanitizer only by breaking the strong unforgeability of the sanitizer UOV instance or by causing an H_J collision.

Game B: malicious sanitizer frames the signer. This case is symmetric. The adversary knows (sk_{San}, tk) but not sk_S . It wins only if the judge outputs **Signer** on a valid tuple not output by the honest signing protocol.

Then necessarily $\omega^* = \text{Signer}$ and $\text{UOV.Verify}(pk_S, d_{\text{Signer}}^*, \tau^*) = 1$ with $d_{\text{Signer}}^* = \text{HashToF}(H_J(m^*, \text{ADM}^*, \sigma_S^*, \sigma_{San}^*, (r_i^*), \text{Signer}))$.

As before, except with probability at most $q_{H_J}^2/2^{\kappa+1}$, either τ^* is a strong UOV forgery under pk_S , or the tuple is identical to an honestly issued signer-origin tuple, contradicting the winning condition. The trapdoor tk does not help because it belongs to an independent UOV instance.

Combining the two games gives the claimed bound.

9 Experimental Evaluation

We implemented UOV-San in C using the reference implementation of UOV [27] as the underlying primitive. All experiments were conducted on a machine with an Intel Core i7-1185G7 processor (3.0 GHz base, 4.8 GHz boost, Tiger Lake microarchitecture) with 16 GB DDR4 RAM, running Ubuntu 22.04 LTS with GCC 11.3 at optimisation level -O3. We used SHA3-256 (via OpenSSL 3.0) for

Table 5: Timing results for UOV-San ($|\text{ADM}| = 4$, message size 1 KB, $|I| = 4$ modified blocks for sanitization). Times in milliseconds (ms); median of 10,000 trials. “Sign” = full interactive protocol (Algorithms 2–4); “San” = sanitization (Algorithm 6).

Level	KeyGen (ms)	Sign (ms)	Verify (ms)	San (ms)
I	12.3	4.8	1.6	3.2
III	38.7	9.4	3.1	6.8
V	91.2	18.6	5.9	13.4

all random oracles (H_S, H_{San}, H_2, H_J). Each measurement is the median of 10,000 trials; standard deviations are below 3% in all cases. We set $|\text{ADM}| = 4$ admissible blocks and message size 1 KB unless otherwise noted.

9.1 Timing Results

Table 5 presents the timing results for UOV-San at all three security levels. The signing time includes both Sign_1 (Algorithm 2) and Sign_2 (Algorithm 3), i.e., the full two-message interactive protocol, plus the tag generation in Sign_3 . In practice, these steps can be pipelined: the signer sends msg_1 immediately after computing it, and the sanitizer processes it concurrently. The round-trip latency is dominated by network delay in distributed deployments.

Verification (Algorithm 5) now includes an additional UOV verification for the accountability tag, making it slightly slower than the original but still very fast. Sanitization (Algorithm 6) also includes one extra UOV signing for the new tag.

9.2 Signature Size vs. Number of Admissible Blocks

Figure 1 (left) shows how signature size grows with the number of admissible blocks $|\text{ADM}|$. With the embedded accountability tag, the signature size now includes three UOV signatures: two for the dual layers and one for the tag. The growth is linear: $|\sigma| = 3 \cdot |\sigma_{\text{UOV}}| + |\text{ADM}| \cdot n + O(|\text{ADM}|)$ bytes, where the $|\text{ADM}| \cdot n$ term accounts for the chameleon randomness $(r_i)_{i \in \text{ADM}}$ (each $r_i \in \mathbb{F}_q^n$ requires n field elements). For Level I ($n = 116$, $|\sigma_{\text{UOV}}| = 116$ bytes), the slope is 116 bytes per admissible block plus a constant overhead of $3 \times 116 = 348$ bytes.

9.3 Sanitization Overhead

Figure 1 (right) shows sanitization time as a function of the number of modified blocks $|I| \leq |\text{ADM}| = 4$. The base cost ($|I| = 0$) is one UOV.Sign call for σ'_{San} plus one UOV.Sign call for the new tag τ' (Algorithm 6, lines 10 and 13); each additional modified block adds one TCTH.TrapColl call (one UOV signing operation, Algorithm 6, line 5). The linear growth confirms that the dominant cost is the UOV signing operations, not the hash computations.

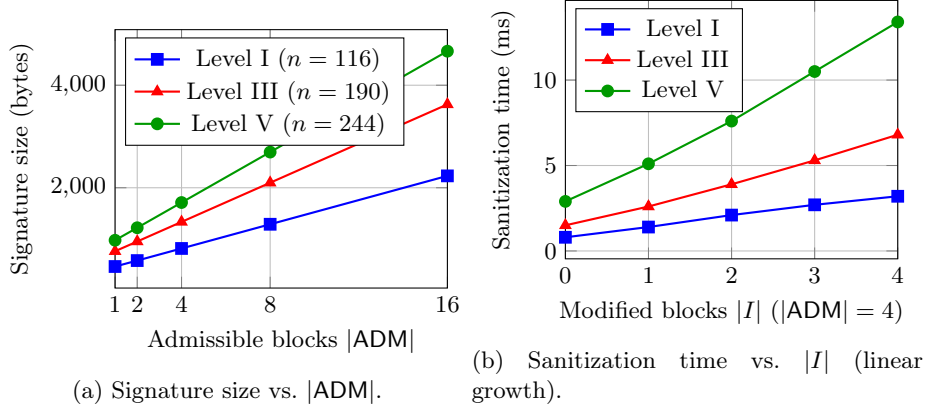


Fig. 1: Performance characteristics of UOV-San.

Table 6: Comparison with prior post-quantum SSS at 128-bit security ($|ADM| = 4$). PK = total public key size; Sig = signature size. “✓”=achieved, “×”=not achieved.

Scheme	PK	Sig	Unf.	Imm.	Acc.	PQ
Ahmad et al. [1]	~1 MB	~4 KB	✓	✓	✓	✓
UOV-San Level I (ours)	~200 KB	~816 B	✓	✓	✓	✓
UOV-San Level III (ours)	~835 KB	~1,334 B	✓	✓	✓	✓

9.4 Comparison with Prior Post-Quantum SSS

Table 6 compares UOV-San with the only prior post-quantum SSS achieving accountability, Ahmad et al. [1]. UOV-San achieves a $\approx 5\times$ reduction in signature size at 128-bit security (816 B vs. 4 KB), at the cost of a larger public key and the deliberate absence of transparency. The public key of UOV-San consists of three UOV public keys (pk_S, pk_{San}, ck); at Level I, this is $3 \times 66.6 \approx 200$ KB.

10 Parameters and Efficiency Summary

Signature size formula. As shown in Figure 1a, with the embedded accountability tag, the signature size becomes

$$|\sigma| = 3 \cdot |\sigma_{UOV}| + |ADM| \cdot n + \delta' \quad \text{bytes,}$$

where the third UOV signature is the accountability tag τ , and $\delta' \approx |ADM| \cdot [\log_2 \ell / 8] + O(1)$ B accounts for encoding of ADM and length fields. For $|ADM| = 4$, $\ell \leq 256$ ($\delta' \approx 4$ B):

- Level I: $3 \times 116 + 4 \times 116 + 4 = 816$ B.
- Level III: $3 \times 190 + 4 \times 190 + 4 = 1,334$ B.
- Level V: $3 \times 244 + 4 \times 244 + 4 = 1,712$ B.

Public key size. The public key is (pk_S, pk_{San}, ck) , three UOV public keys. As shown in Table 1:

- Level I: $3 \times 66.6 \approx 200$ KB.
- Level III: $3 \times 278.4 \approx 835$ KB.
- Level V: $3 \times 838 \approx 2,514$ KB.

Communication cost. The interactive signing protocol (Algorithms 2–4) requires one round-trip:

$$|\text{msg}_1| = |m| + |\text{ADM}| + |\sigma_S| + |\text{ADM}| \cdot (n + \kappa/8) \text{ bytes}; \quad |\text{msg}_2| = |\sigma_{San}| \text{ bytes}.$$

For Level I, $|\text{ADM}| = 4$, $\kappa = 256$: $|\text{msg}_1| \approx |m| + 4 \times (116 + 32) = |m| + 592$ B; $|\text{msg}_2| = 116$ B.

Hash function instantiation. We use four independent random oracles H_S, H_{San}, H_2, H_J , each instantiated as SHA3-256 with a distinct domain-separation prefix. The $\kappa = 256$ -bit output ensures birthday resistance at 128-bit security. The additional mapping HashToF is implemented by taking the first $o \cdot \log_2 q$ bits of SHAKE256 output and interpreting them as a vector in $\mathbb{F}_q^o$.

11 Conclusion

We presented UOV-San, a post-quantum sanitizable signature scheme based on multivariate cryptography and a dedicated target-collision trapdoor hash, MV-CH. The scheme achieves unforgeability, immutability, and accountability—including accountability against a malicious signer—under UOV EUF-CMA security and target second-preimage resistance (tSPR) in the random-oracle model, as proved in Theorems 4–6. We forgo transparency and privacy. As explained in Section 7, these properties are structurally incompatible with the dual-signature architecture required for key separation and true accountability: preserving σ_S across sanitizations (necessary for correctness without signer participation) makes σ_S a fingerprint detectable by any verifier. We precisely characterise the application domains where this trade-off is acceptable, including supply chain audit, government document redaction, and blockchain audit trails. Experimental evaluation (Section 9) confirms practical performance: signing under 5 ms, verification under 2 ms, and sanitization under 3.5 ms at Level I on commodity hardware. Signature sizes of ≈ 816 B at Level I represent a $\approx 5\times$ improvement over the only prior accountable post-quantum SSS [1].

Some future directions. Achieving transparency in a post-quantum SSS without sacrificing key separation remains an open problem. Potential approaches include zero-knowledge proofs of sanitization equivalence or alternative multivariate structures where σ_S can be re-randomised without exposing sk_S . Reducing the public key size (currently ≈ 200 KB at Level I) via compact multivariate representations is another direction. Extending the scheme to support unlinkability [7] or invisibility [8] in the post-quantum setting is also of interest.

References

1. Ahmad, S., Rass, S., Seyedi, Z.: Post-quantum sanitizable signatures from McEliece-based chameleon hashing. arXiv:2602.20657 [cs.CR] (2026)
2. Ateniese, G., Chou, D.H., de Medeiros, B., Tsudik, G.: Sanitizable signatures. In: ESORICS 2005. LNCS, vol. 3679, pp. 159–177. Springer (2005)
3. Bardet, M., Faugère, J.-C., Salvy, B.: On the complexity of Gröbner basis computation of semi-regular overdetermined algebraic equations. In: Proc. ICPSS 2004, pp. 71–74 (2004)
4. Beullens, W.: Improved cryptanalysis of UOV and Rainbow. In: EUROCRYPT 2021. LNCS, vol. 12696, pp. 348–373. Springer (2021)
5. Brassard, G., Høyer, P., Tapp, A.: Quantum algorithm for the collision problem. ACM SIGACT News 28(2), 14–19 (1997)
6. Brzuska, C., Fischlin, M., Freudenreich, T., Lehmann, A., Page, M., Schelbert, J., Schröder, D., Volk, F.: Security of sanitizable signatures revisited. In: PKC 2009. LNCS, vol. 5443, pp. 317–336. Springer (2009)
7. Brzuska, C., Fischlin, M., Lehmann, A., Schröder, D.: Unlinkability of sanitizable signatures. In: PKC 2010. LNCS, vol. 6056, pp. 444–461. Springer (2010)
8. Camenisch, J., Drijvers, M., Lehmann, A., Serrano, A., Yung, M.: Invisible sanitizable signatures and public-key encryption are equivalent. In: PKC 2017. LNCS, vol. 10175, pp. 630–660. Springer (2017)
9. Casanova, A., Faugère, J.-C., Macario-Rat, G., Patarin, J., Perret, L., Ryckeghem, J.: GeMSS: A great multivariate short signature. NIST PQC Submission (2017)
10. Courtois, N., Klimov, A., Patarin, J., Shamir, A.: Efficient algorithms for solving overdefined systems of multivariate polynomial equations. In: EUROCRYPT 2000. LNCS, vol. 1807, pp. 392–407. Springer (2000)
11. Derler, D., Slamanig, D., Striecks, C., Tairi, E.: Rethinking privacy for extended sanitizable signatures and a black-box construction of strongly private schemes. In: ESORICS 2019. LNCS, vol. 11736, pp. 455–474. Springer (2019)
12. Ding, J., Schmidt, D.: Rainbow, a new multivariable polynomial signature scheme. In: ACNS 2005. LNCS, vol. 3531, pp. 164–175. Springer (2005)
13. Garey, M.R., Johnson, D.S.: Computers and Intractability: A Guide to the Theory of NP-Completeness. W.H. Freeman (1979)
14. Liu, R., Lu, Y., Pan, J.: Flexible Non-interactive Short-term Implicit Certificate Generation for VANETs. arXiv preprint arXiv:2402.02607 (2024)
15. Kipnis, A., Patarin, J., Goubin, L.: Unbalanced oil and vinegar signature schemes. In: EUROCRYPT 1999. LNCS, vol. 1592, pp. 206–222. Springer (1999)
16. Kipnis, A., Shamir, A.: Cryptanalysis of the oil and vinegar signature scheme. In: CRYPTO 1998. LNCS, vol. 1462, pp. 257–266. Springer (1998)
17. Krawczyk, H., Rabin, T.: Chameleon signatures. In: NDSS 2000. Internet Society (2000)
18. Krenn, S., Slamanig, D., Striecks, C.: Practical witness-hiding sanitizable signatures. In: ESORICS 2019. LNCS, vol. 11736, pp. 425–454. Springer (2019)
19. Lai, Q., Liu, F., Ma, Z.: Lattice-based sanitizable signatures. In: ACISP 2019. LNCS, vol. 11547, pp. 359–374. Springer (2019)
20. National Institute of Standards and Technology: Post-Quantum Cryptography Standardization. <https://csrc.nist.gov/projects/post-quantum-cryptography> (2024)
21. Patarin, J.: The oil and vinegar algorithm for signatures. Presented at the Dagstuhl Workshop on Cryptography (1997)

22. Shor, P.W.: Algorithms for quantum computation: Discrete logarithms and factoring. In: Proc. FOCS 1994, pp. 124–134. IEEE (1994)
23. Simon, D.R.: Finding collisions on a one-way street: Can secure hash functions be based on general assumptions? In: EUROCRYPT 1998. LNCS, vol. 1403, pp. 334–345. Springer (1998)
24. Ateniese, G., Magri, B., Venturi, D., Andrade, E.: Redactable blockchain—or—rewriting history in Bitcoin and friends. In: EuroS&P 2017, pp. 111–126. IEEE (2017)
25. Sanders, O. (2020). Efficient Redactable Signature and Application to Anonymous Credentials. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds) Public-Key Cryptography – PKC 2020. PKC 2020. Lecture Notes in Computer Science(), vol 12111. Springer, Cham.
26. Slamanig, D., Striecks, C., Tairi, E.: Highly-efficient fully-anonymous dynamic group signatures. In: AsiaCCS 2019. ACM (2019)
27. Beullens, W., Faugère, J.-C., Koussa, E., Macario-Rat, G., Patarin, J., Perret, L.: UOV: Unbalanced Oil and Vinegar. NIST PQC Round 1 Additional Signatures Submission (2022). <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>